
Software Requirements Specification (SRS)

Plataforma de Gestión de Eventos Académicos

Integrantes: Sebastian Eduardo Fanchi Lopez
Juan Diego Peña Mayorga

Versión: 3.0

Fecha: 11 de mayo de 2026

Estado: Versión final — Entrega 3

Organización: Pontificia Universidad Javeriana

Curso: Diseño de Software Basado en Patrones

Control de cambios

Versión	Fecha	Autor	Descripción	Estado
0.1	04/03/2026	S. Fanchi J. Peña	Estructura base del SRS: propósito y alcance, stakeholders con necesidades, lista preliminar de 20 RF y borrador de RNF por atributos de calidad; referencia inicial al modelo ER.	Borrador
0.2	08/03/2026	S. Fanchi J. Peña	Ampliación a 30 RF con priorización MoSCoW; criterios de aceptación verificables; RNF extendidos a 15–18; CU-01 a CU-06 con flujos alternativos.	Borrador
0.3	15/03/2026	S. Fanchi J. Peña	Consolidación inicial: estructura base completa, 30 RF, 20 RNF preliminares, 6 casos de uso y modelo ER referenciado.	Borrador
0.4	18/03/2026	S. Fanchi J. Peña	Sección 2 (descripción general), CU-07, reglas en tabla, RNF de testeabilidad y observabilidad, modelo ER en TikZ, matriz de trazabilidad y definiciones.	Borrador
0.5	28/03/2026	S. Fanchi J. Peña	Limpieza de ambigüedades en RF críticos, normalización de criterios de aceptación y revisión MoSCoW con foco en entregables de corto plazo.	Borrador
0.6	02/04/2026	S. Fanchi J. Peña	Casos de uso con variantes de error y validaciones; alineación entre actores, flujos y reglas de negocio.	Borrador
0.7	06/04/2026	S. Fanchi J. Peña	Refinamiento de RNF de rendimiento, disponibilidad y seguridad; métricas operativas base para pruebas de carga.	Borrador
0.8	10/04/2026	S. Fanchi J. Peña	Incorporación de requisitos de observabilidad (logs estructurados, métricas RED, trazas distribuidas) y trazabilidad entre stakeholder, requisito y componente.	Borrador
0.9	14/04/2026	S. Fanchi J. Peña	Consistencia con el SAD: dependencias externas, alcance por fases, riesgos de integración y criterios de aceptación sistémicos.	Borrador
0.10	17/04/2026	S. Fanchi J. Peña	Matriz de trazabilidad completa y cierre editorial de definiciones, glosario y restricciones de cumplimiento.	Borrador
0.11	20/04/2026	S. Fanchi J. Peña	Revisión preentrega y verificación de evidencias para demo.	Borrador
1.0	21/04/2026	S. Fanchi J. Peña	Entrega 1: RF/RNF cerrados, trazabilidad validable y énfasis en observabilidad.	Entregado
2.0	23/04/2026	S. Fanchi J. Peña	Entrega 2: tabla de niveles de integración externa (L1–L3), RN-11 (observabilidad transversal obligatoria), glosario ampliado y alineación con el prototipo.	Entregado
2.1	24/04/2026	S. Fanchi J. Peña	Operacionalización completa de Ley 1581, tabla de riesgos RI-01–RI-05 y escenarios de carga diferenciados.	Aprobado

Versión	Fecha	Autor	Descripción	Estado
3.0	11/05/2026	S. Fanchi J. Peña	Entrega 3: consolidación de RNF (28 a 18), priorización MoSCoW revisada con requisito <i>Won't have</i> explícito (CFP), criterios de aceptación reforzados con referencia a <code>pytest</code> y <code>Locust</code> , RF-30 ampliado con búsqueda semántica (<code>pgvector</code>) elevado a <i>Must</i> , reescritura en primera persona de secciones contextuales y ajustes de layout.	Entregado

Índice

1. Introducción	3
1.1. Propósito y contexto del documento	3
1.2. Alcance del sistema	3
1.3. Definiciones, acrónimos y abreviaturas	4
1.4. Referencias	5
1.5. Visión general del documento	5
2. Descripción general del sistema	5
2.1. Perspectiva del producto	5
2.2. Funciones del producto	5
2.3. Características de los usuarios	6
2.4. Restricciones	6
2.5. Supuestos y dependencias	7
2.6. Nivel de integración con sistemas externos	7
3. Stakeholders y necesidades	9
3.1. Identificación de stakeholders	9
3.2. Priorización de necesidades	9
4. Requisitos funcionales (RF)	9
4.1. Priorización MoSCoW	9
4.2. Catálogo de requisitos funcionales	9
5. Requisitos no funcionales (RNF)	13
5.1. Observabilidad	13
5.2. Rendimiento	15
5.3. Escalabilidad	15
5.4. Disponibilidad	15
5.5. Seguridad y privacidad	16
5.6. Usabilidad	16
5.7. Interoperabilidad	17
5.8. Modificabilidad	17
5.9. Testeabilidad	18
6. Reglas de negocio	18
7. Casos de uso	20
7.1. CU-01: Inscribirse a un evento	20
7.2. CU-02: Crear y publicar un evento	20

7.3. CU-03: Enviar comunicación masiva a inscritos	21
7.4. CU-04: Descargar certificado digital	21
7.5. CU-05: Gestionar invitación a ponente	22
7.6. CU-06: Supervisión operacional y diagnóstico	22
7.7. CU-07: Aprobación institucional de eventos	22
8. Riesgos de implementación	23
9. Modelo de datos conceptual	24
9.1. Diagrama Entidad-Relación	24
9.2. Descripción de entidades principales	24
10. Matriz de trazabilidad	25
10.1. Trazabilidad primaria: Stakeholder → Requisito	25
10.2. Trazabilidad extendida de requisitos críticos	26
11. Criterios de aceptación del sistema	27

1. Introducción

1.1. Propósito y contexto del documento

Este documento especifica los requisitos del sistema que diseñamos como respuesta al problema de fragmentación en la gestión de eventos académicos de la Pontificia Universidad Javeriana. Hoy cada facultad, departamento o grupo estudiantil organiza sus eventos con herramientas heterogéneas —hojas de cálculo, correos, formularios de Google o sistemas internos—, lo que genera duplicación de esfuerzos, falta de visibilidad institucional, problemas de escalabilidad ante eventos masivos y experiencias inconsistentes para los asistentes.

Nuestra propuesta es la **Plataforma de Gestión de Eventos Académicos (PGEA)**, una solución centralizada que reemplaza ese ecosistema fragmentado. Adicionalmente, escogimos la **observabilidad** como atributo de calidad principal del proyecto: diseñamos el sistema desde el primer sprint con instrumentación de logs, métricas y trazas, de forma que cada flujo crítico sea operacionalmente verificable y diagnosticable.

El documento sigue la norma ISO/IEC/IEEE 29148:2018 y actúa como contrato técnico entre nosotros (el equipo de desarrollo), el cliente institucional y los evaluadores del curso. Por la naturaleza académica del proyecto, la implementación es un POC funcional: los servicios externos (pasarela de pagos, proveedor de correo, SSO) se conectan mediante adaptadores con modos mock o sandbox, sin que ello afecte la validez del diseño.

Audiencia objetivo:

- **Desarrolladores:** referencia técnica para la implementación.
- **Arquitectos:** base para las decisiones del SAD.
- **Evaluadores del curso:** verificación de completitud y calidad de requisitos.
- **Cliente / institución:** validación de que el sistema responde a sus necesidades.

1.2. Alcance del sistema

La PGEA centraliza la planificación, promoción, gestión, ejecución y análisis de eventos académicos (conferencias, seminarios, talleres, congresos, simposios) de la Universidad Javeriana. Atiende a cinco perfiles —organizadores, asistentes, ponentes, revisores y administradores— e integra los servicios institucionales existentes mediante adaptadores con niveles de automatización explícitamente definidos (Tabla 4).

El sistema se construye como un **POC funcional**: implementamos completamente todos los RF clasificados como *Must have* (M); los *Should have* (S) se implementan si el cronograma lo permite; los *Could have* (C) quedan especificados como contrato de API y se demostrarán parcialmente; los *Won't have* (W) están documentados solo como referencia para una fase posterior.

Fuera del alcance: gestión de espacios físicos y reservas de salones, streaming de video en vivo (nos integramos con Zoom/Teams mediante enlace externo), nómina o contratación de ponentes, evaluación de desempeño docente y gestión de proyectos de investigación asociados a eventos.

1.3. Definiciones, acrónimos y abreviaturas

Término	Definición
SRS	<i>Software Requirements Specification</i> . Especificación de Requisitos de Software.
SAD	<i>Software Architecture Document</i> . Documento de Arquitectura de Software.
RF	Requisito Funcional.
RNF	Requisito No Funcional.
RN	Regla de Negocio.
MoSCoW	Técnica de priorización: <i>Must have</i> , <i>Should have</i> , <i>Could have</i> , <i>Won't have</i> .
PGEA	Plataforma de Gestión de Eventos Académicos.
POC	<i>Proof of Concept</i> . Prototipo funcional que valida el diseño.
SSO	<i>Single Sign-On</i> . Autenticación delegada a un proveedor de identidad.
OAuth 2.0	Protocolo estándar de autorización delegada (RFC 6749).
OpenID Connect	Capa de identidad sobre OAuth 2.0 para autenticación federada.
RBAC	<i>Role-Based Access Control</i> . Control de acceso basado en roles.
iCal / ICS	Formato estándar de intercambio de datos de calendario (RFC 5545).
p95	Percentil 95 de tiempos de respuesta en pruebas de carga.
RTO	<i>Recovery Time Objective</i> . Tiempo máximo de recuperación ante fallo.
RPO	<i>Recovery Point Objective</i> . Pérdida máxima aceptable de datos ante fallo.
TLS	<i>Transport Layer Security</i> . Protocolo de cifrado en tránsito.
PII	<i>Personally Identifiable Information</i> . Datos personales sujetos a Ley 1581.
Ley 1581/2012	Ley colombiana de Protección de Datos Personales.
CFP	<i>Call for Papers</i> . Convocatoria abierta para propuestas académicas.
OTel	<i>OpenTelemetry</i> . Framework de observabilidad neutral al proveedor (CNCF).
OTLP	<i>OpenTelemetry Protocol</i> . Protocolo de exportación de telemetría.
RED	Método de monitoreo basado en <i>Rate</i> , <i>Errors</i> y <i>Duration</i> .
SLI / SLO	<i>Service Level Indicator / Objective</i> . Métrica y umbral de calidad operacional.
W3C Trace	Estándar W3C <i>Trace Context</i> para propagación de identificadores de trazabilidad.

Término	Definición
Prometheus	Sistema de monitoreo de métricas open-source (CNCF).
Grafana	Plataforma de visualización de métricas, trazas y logs.
Loki	Agregador de logs de Grafana Labs.
Jaeger / Tempo	Backends de trazas distribuidas compatibles con OTel.
pgvector	Extensión de PostgreSQL para embeddings vectoriales y búsqueda semántica.
pytest	Framework de testing para Python (unitario e integración).
Locust	Herramienta open-source de pruebas de carga escrita en Python.

Tabla 2: Definiciones, acrónimos y abreviaturas.

1.4. Referencias

- ISO/IEC/IEEE 29148:2018 — *Systems and software engineering — Life cycle processes — Requirements engineering*.
- ISO/IEC/IEEE 42010:2011 — *Systems and software engineering — Architecture description*.
- Ley 1581 de 2012 — Protección de Datos Personales, Colombia.
- Documento de proyecto: *PROYECTO — Diseño de Software Basado en Patrones, Propuesta Plataforma de Gestión de Eventos Académicos*, 2026, luis.barreto@javeriana.edu.co.
- SAD de la PGEA — mismo repositorio del proyecto.

1.5. Visión general del documento

El documento se organiza en once secciones: la Sección 2 describe el contexto general del sistema e incluye la tabla de niveles de integración con sistemas externos; la Sección 3 identifica a los interesados y sus necesidades; la Sección 4 especifica los 30 requisitos funcionales con priorización MoSCoW y criterios de aceptación verificables; la Sección 5 presenta los 18 requisitos no funcionales organizados por atributo de calidad, comenzando por observabilidad como eje central del diseño; la Sección 6 documenta las reglas de negocio del dominio; la Sección 7 detalla los casos de uso principales con flujos alternativos; la Sección 8 consolida los riesgos de implementación; la Sección 9 presenta el modelo de datos conceptual; la Sección 10 contiene la trazabilidad primaria y la trazabilidad extendida de los requisitos críticos; y la Sección 11 cierra con los criterios de aceptación del sistema.

2. Descripción general del sistema

2.1. Perspectiva del producto

La PGEA es un sistema **nuevo e independiente** que reemplaza los procesos fragmentados actuales por una solución centralizada. No es una evolución de un sistema previo; coexiste con los sistemas institucionales existentes mediante integraciones bien definidas, sin depender de ninguno para su núcleo funcional.

2.2. Funciones del producto

Los bloques funcionales del sistema son:

1. **Catálogo y descubrimiento de eventos:** listado con filtros combinables, búsqueda textual con autocompletado y búsqueda semántica basada en embeddings vectoriales (**pgvector**).

2. **Gestión del ciclo de vida del evento:** creación, edición, publicación, seguimiento y cancelación, con aprobación institucional opcional.
3. **Inscripciones y pagos:** flujo completo de inscripción (gratuita y paga), integración con pasarelas mediante adaptador mock, control atómico de cupos concurrentes.
4. **Comunicaciones y notificaciones:** envío asíncrono de correos mediante cola, confirmaciones automáticas y recordatorios configurables.
5. **Agenda y ponentes:** gestión de sesiones, tracks e invitaciones a conferencistas con material de presentación.
6. **Certificados digitales:** generación, emisión y verificación de certificados de asistencia, ponencia y organización.
7. **Analítica y reportes:** dashboards por evento (organizador) y dashboards institucionales (administrador).
8. **Administración y seguridad:** gestión de usuarios, roles y permisos; aprobación institucional de eventos; auditoría inmutable de acciones críticas.
9. **Observabilidad transversal:** instrumentación con OpenTelemetry de todos los flujos críticos mediante logs estructurados, métricas en tiempo real (método RED) y trazabilidad distribuida. Este bloque es el foco arquitectónico del proyecto.

2.3. Características de los usuarios

Tipo de usuario	Nivel técnico	Frecuencia	Consideraciones
Organizador	Medio	Ocasional (por semestre)	Docente, coordinador o líder estudiantil. Sin experiencia previa con herramientas de gestión de eventos; requiere flujo guiado con valores por defecto claros.
Asistente	Básico	Variable (por evento)	60 % desktop, 35 % móvil, 5 % tablet. Incluye usuarios externos sin cuenta institucional.
Ponente	Medio	Muy ocasional	Puede ser externo sin SSO; accede mediante token de invitación de un solo uso.
Revisor	Alto	Episódico (por convocatoria)	Perfil académico familiarizado con sistemas tipo EasyChair.
Administrador	Alto	Regular (semanal)	Personal de TI o coordinación institucional.

Tabla 3: Características de los usuarios del sistema.

2.4. Restricciones

1. **Regulatoria:** cumplimiento obligatorio de la Ley 1581 de 2012 para el tratamiento de datos personales de usuarios.
2. **Temporal:** desarrollo e implementación en 16 semanas del semestre académico.
3. **Presupuestaria:** costo de infraestructura máximo de \$200 USD/mes; en el POC trabajamos con una instancia única de AWS Lightsail.
4. **Organizacional:** la autenticación debe integrarse con el sistema institucional (Google Workspace / SSO).
5. **Tecnológica:** adoptamos un stack **open-source basado en Python** por afinidad del equipo. Backend en Python; pruebas unitarias e integración con **pytest**; pruebas de carga con **Locust**; base de datos PostgreSQL con extensión **pgvector**; observabilidad con OpenTelemetry, Prometheus, Grafana, Loki y Jaeger/Tempo; despliegue con Docker sobre

AWS Lightsail con Nginx como reverse proxy; CI/CD con GitHub Actions en repositorio monorepo (rama **main** para producción, rama **stage** para validación).

6. **Fuera del alcance:** gestión de espacios físicos, streaming de video en vivo, nómina de ponentes, evaluación de desempeño docente, gestión de proyectos de investigación.
7. **Priorización de alcance:** garantizamos la implementación de todos los RF *Must have* y los RNF de observabilidad. Los *Should have* se incorporan según disponibilidad de tiempo; los *Could have* quedan como contrato de API.

2.5. Supuestos y dependencias

1. Tenemos disponible infraestructura cloud (instancia AWS Lightsail) para el despliegue del POC.
2. La fase 1 institucional estimaría 500 organizadores, 20.000 asistentes y 100–150 eventos por semestre, con crecimiento del 20–30 % anual.
3. Los usuarios tienen acceso a internet; no se requiere modo offline.
4. La universidad usa Google Workspace para correo institucional (@javeriana.edu.co).
5. En el POC, los servicios externos (pasarela de pagos, proveedor de correo, SSO) se conectan mediante adaptadores con modo mock o sandbox; los adaptadores son reemplazables por integraciones reales sin modificar la lógica de negocio.
6. Los usuarios externos (ponentes sin cuenta institucional) acceden mediante tokens de invitación de un solo uso.

2.6. Nivel de integración con sistemas externos

Las integraciones de la PGEA se clasifican en tres niveles de automatización. Esta clasificación cierra la ambigüedad entre una integración meramente referencial y una completamente orquestada, lo que tiene impacto directo en la complejidad de implementación y en los riesgos del POC.

Sistema externo	Nivel	Descripción del grado de integración
Autenticación SSO (Google / MS 365)	L3 — Orquestada	Delegación completa vía OAuth 2.0 / OpenID Connect. En el POC usamos un adaptador mock que replica el comportamiento del proveedor real y puede sustituirse por la integración real sin modificar la lógica de negocio.
Pasarela de pagos (PSE / tarjetas)	L2 — Semiautomática	Redirección al proveedor (no gestionamos datos de tarjeta); confirmación asíncrona vía webhook; ciclo de estados con idempotencia. En el POC usamos un mock que simula los webhooks de la pasarela.
Correo electrónico (proveedor SMTP/API)	L2 — Semiautomática	Envío asíncrono encolado mediante el adaptador. En el POC, sin clave de proveedor configurada, los intentos quedan registrados en los logs estructurados como evidencia operacional.
Videoconferencia (Zoom / Teams)	L1 — Referencial	El organizador configura un enlace externo. La PGEA almacena y controla el acceso al enlace (solo inscritos confirmados), pero no crea ni gestiona reuniones vía API.
Calendario personal (Google Cal / Outlook)	L1 — Referencial	Generamos un archivo <code>.ics</code> (RFC 5545) descargable. No realizamos sincronización bidireccional.

Tabla 4: Clasificación del nivel de integración con sistemas externos. L1 = Referencial, L2 = Semiautomática, L3 = Orquestada.

3. Stakeholders y necesidades

3.1. Identificación de stakeholders

Nombre / Rol	ID	Necesidades principales	Prior.
Organizador	STK-01	Crear y publicar eventos; configurar inscripciones (gratuitas y pagas); gestionar inscritos en tiempo real; enviar comunicaciones masivas; generar certificados; acceder a reportes y estadísticas por evento.	Alta
Asistente	STK-02	Descubrir y filtrar eventos; inscribirse fácilmente; recibir confirmaciones y recordatorios automáticos; acceder al material del evento; descargar certificados digitales.	Alta
Ponente	STK-03	Recibir invitaciones formales; confirmar o declinar participación; completar perfil público; subir material de presentación.	Media
Revisor	STK-04	Recibir asignación de propuestas; evaluar con rúbrica; dar feedback al autor; emitir recomendación de aceptación o rechazo.	Media
Administrador	STK-05	Supervisar todos los eventos; aprobar o rechazar publicaciones; acceder a dashboards institucionales (incluyendo señales de observabilidad); gestionar usuarios y permisos.	Alta
Universidad	STK-06	Vista unificada de la actividad académica; métricas institucionales consolidadas; cumplimiento verificable de la Ley 1581; control de calidad de los eventos publicados a nombre de la institución.	Alta

Tabla 5: Stakeholders, necesidades y prioridad.

3.2. Priorización de necesidades

Las necesidades de organizadores (STK-01) y asistentes (STK-02) son críticas porque definen el núcleo funcional del sistema. Las de ponentes (STK-03) y revisores (STK-04) aplican a eventos con agenda formal o convocatoria académica y tienen prioridad media. Las de administrador (STK-05) y universidad (STK-06) condicionan la arquitectura completa: seguridad, integración SSO, analítica institucional, observabilidad y cumplimiento de la Ley 1581.

4. Requisitos funcionales (RF)

4.1. Priorización MoSCoW

M = *Must have* (obligatorio en la demo final). **S** = *Should have* (importante; se implementa si el cronograma lo permite). **C** = *Could have* (deseable; se especifica el contrato y se demuestra parcialmente si hay tiempo). **W** = *Won't have* (fuera del alcance del POC; documentado solo como referencia).

Cada RF tiene su criterio de aceptación verificable. Salvo indicación contraria, la verificación se realiza mediante `pytest` (unitario o integración) en el pipeline CI de GitHub Actions, e incluye casos felices y casos de error.

4.2. Catálogo de requisitos funcionales

ID	Descripción	M	Criterio de aceptación	STK / Verif.
RF-01	Listar eventos con filtros combinables: fecha, categoría, modalidad y texto libre.	M	Con tres filtros activos simultáneamente, <code>GET /events</code> retorna solo los eventos que cumplen los criterios; si el resultado es vacío, responde 200 con array vacío (no 404); verificado con <code>pytest</code> cubriendo combinaciones válidas y vacías.	STK-02 pytest
RF-02	Ordenar eventos por fecha (ascendente/descendente) y nombre alfabético.	M	Los criterios producen resultados correctamente ordenados sobre cualquier conjunto; el orden persiste al combinar con filtros; verificado con <code>pytest</code> sobre dataset de prueba.	STK-02 pytest
RF-03	Inscripción gratuita a evento con control atómico de cupos.	M	Con token válido y cupo > 0: <code>POST /events/{id}/register</code> retorna 201; el cupo se decrementa atómicamente en 1 (transacción con bloqueo pesimista); sin token retorna 401; sin cupo retorna 409 con <code>no_capacity</code> ; inscripción duplicada retorna 409 con <code>duplicate_registration</code> ; verificado con <code>pytest</code> (caso feliz, sin cupo, duplicado) y prueba de concurrencia con <code>Locust</code> simulando 50 inscripciones simultáneas sin sobreventa.	STK-02 pytest + Locust
RF-04	Inscripción a eventos de pago mediante adaptador de pasarela (mock en POC).	M	(a) Al iniciar el pago: cupo reservado en estado <code>pendiente_pago</code> , retornando URL del adaptador mock. (b) Webhook de confirmación positivo: estado pasa a <code>confirmada</code> , correo asíncrono encolado. (c) Webhook ausente al expirar el timeout (15 min configurable): cupo liberado, estado <code>expirada</code> . (d) Webhook duplicado: respuesta idempotente sin doble procesamiento. Verificado con <code>pytest</code> cubriendo los cuatro estados usando el servidor mock de webhooks.	STK-02 pytest
RF-05	Crear eventos con título, descripción, fecha/hora, modalidad, ubicación o enlace, y cupo máximo.	M	<code>POST /events</code> retorna 201 con el evento en estado <code>borrador</code> y el id asignado; sin token: 401; campo obligatorio ausente: 400 con mensaje específico; fecha de fin anterior a inicio: 400; cupo < 1: 400; el evento creado es recuperable inmediatamente; verificado con <code>pytest</code> (cinco escenarios).	STK-01 pytest
RF-06	Editar, publicar y cancelar un evento creado.	M	<i>Edición</i> : solo el organizador dueño puede actualizar; tercero recibe 403. <i>Publicación</i> : evento pasa de <code>borrador</code> a <code>publicado</code> y aparece en el catálogo en ≤ 10 s. <i>Cancelación</i> : estado <code>cancelado</code> ; inscritos confirmados reciben correo asíncrono; operación registrada en log de auditoría (RF-29); verificado con <code>pytest</code> .	STK-01 pytest
RF-07	Configurar tipos de inscripción por evento: gratuita, paga y con descuentos por categoría de usuario.	S	Al menos un tipo de inscripción configurable; precio aplicado al inscribirse coincide con el configurado para la categoría del usuario; verificado con <code>pytest</code> para los tres tipos.	STK-01 pytest
RF-08	Definir formularios de inscripción personalizados con campos obligatorios y opcionales por evento.	C	Caso feliz: datos persistidos y vinculados a la inscripción; campo obligatorio vacío: 400 con mensaje específico por campo; datos exportables en el CSV de inscritos; contrato de API documentado.	STK-01 pytest

ID	Descripción	M	Criterio de aceptación	STK / Verif.
RF-09	Crear y gestionar la agenda del evento: sesiones, tracks y horarios.	S	El organizador agrega sesiones con título, fecha/hora inicio y fin, y track; dos sesiones del mismo track que se solapen en tiempo retornan 409 con detalle del conflicto; cambios visibles en la vista pública del evento; verificado con pytest (caso feliz y conflicto).	STK-01 pytest
RF-10	Invitar ponentes y gestionar la aceptación o declinación de su participación.	S	Invitación con enlace de acción de un solo uso; el ponente acepta o declina; el organizador recibe notificación del cambio de estado; enlace expirado o ya utilizado retorna 410; verificado con pytest simulando los tres escenarios.	STK-01, STK-03 pytest
RF-11	Listar los inscritos al evento, con búsqueda por nombre o correo y exportación a CSV.	M	Disponible para el organizador dueño; filtrado funcional sobre el conjunto completo; exportación CSV incluye nombre, correo, estado, fecha de inscripción y datos del formulario personalizado si existen; verificado con pytest comparando el CSV generado contra el conjunto esperado.	STK-01 pytest
RF-12	Enviar comunicaciones masivas por correo a los inscritos de un evento, con segmentación opcional por estado de inscripción.	M	Envío encolado de forma asíncrona; el hilo principal del servidor no se bloquea; estado de entrega por destinatario consultable; la cola expone métricas de tamaño y tasa de error vía RNF-02; en el POC, el adaptador mock registra los envíos en logs estructurados; verificado con pytest (envío encolado y métricas de cola).	STK-01 pytest
RF-13	Generar certificados digitales de asistencia, ponencia y organización con plantilla configurable por evento.	M	Plantilla configurable; generación individual disponible bajo demanda; generación en lote asíncrona disponible para el organizador; cada certificado incluye un código único verificable; sin asistencia registrada (RN-05), la generación retorna 403; verificado con pytest .	STK-01, STK-02 pytest
RF-14	Consultar reportes y estadísticas del evento: inscritos por estado, tasa de ocupación y asistencia por sesión.	S	Dashboard disponible solo para el organizador dueño; expone: total de inscritos por estado, tasa de ocupación, distribución de asistencia por sesión si hay agenda; datos con latencia ≤ 1 min; verificado con pytest contra dataset de prueba.	STK-01 pytest
RF-15	Enviar confirmaciones automáticas de inscripción y recordatorios configurables.	M	Correo de confirmación encolado en ≤ 1 s tras la inscripción (entrega asíncrona); recordatorio enviado dentro del intervalo configurado con margen ≤ 5 min; cada envío genera entrada en logs estructurados con el trace_id de la inscripción origen; verificado con pytest usando adaptador mock.	STK-02 pytest
RF-16	Exportar un evento al calendario personal del asistente.	C	Archivo .ics (RFC 5545) descargable desde la ficha pública; campos correctos: SUMMARY , DTSTART , DTEND , LOCATION , DESCRIPTION ; compatible con Google Calendar y Outlook según prueba manual.	STK-02 Manual
RF-17	Cancelar una inscripción según la política de cancelación del evento.	M	Con inscripción confirmada y dentro del plazo: estado pasa a cancelada , cupo se incrementa atómicamente en 1, y si hay pago asociado se inicia el flujo de reembolso según la política del evento; fuera del plazo: el sistema informa la restricción y no permite cancelación con reembolso; verificado con pytest (dentro y fuera del plazo).	STK-02 pytest

ID	Descripción	M	Criterio de aceptación	STK / Verif.
RF-18	Acceder al material del evento solo con inscripción confirmada.	M	Con confirmada : acceso a presentaciones, agenda completa y enlace de videoconferencia. Con cualquier otro estado: el material privado y el enlace de videoconferencia no son visibles; agenda pública sí visible. Sin inscripción: solo información pública; verificado con pytest para los tres casos.	STK-02, STK-03 pytest
RF-19	Registrar la asistencia de usuarios a un evento o sesión.	M	El organizador marca asistencia individual o importa CSV; si el usuario no tiene inscripción confirmada , el sistema rechaza el registro con 422; la asistencia es condición necesaria para la emisión de certificados (RN-05); verificado con pytest .	STK-01, STK-02 pytest
RF-20	Evaluar el evento tras su finalización mediante un formulario de valoración.	C	Formulario disponible solo para usuarios con asistencia registrada; resultados agregados visibles al organizador; respuestas individuales anonimizadas; contrato de API documentado.	STK-02 pytest
RF-21	Descargar el certificado digital con código de verificación único.	M	Con asistencia registrada (RN-05): PDF con nombre, evento, fecha y código único; enlace válido por al menos 30 días; el código es validable mediante URL pública. Sin asistencia: 403 con mensaje explicativo. Verificado con pytest .	STK-02, STK-03 pytest
RF-22	Completar el perfil público del ponente y subir material de presentación.	S	El ponente actualiza nombre, fotografía (JPEG/PNG, máx. 2 MB), biografía (máx. 500 caracteres) y material (PDF/PPTX, máx. 50 MB); validación específica por tipo y tamaño; perfil visible solo cuando el ponente está confirmado ; verificado con pytest .	STK-03 pytest
RF-23	Recibir asignación de propuestas, evaluarlas con rúbrica y enviar feedback (revisión por pares / CFP).	W	Funcionalidad académica de alta complejidad. Se documenta como referencia para una fase posterior. No implementada en el POC.	STK-04 N/A
RF-24	Supervisar todos los eventos de la institución y aprobar o rechazar publicaciones con comentario.	S	Listado global filtrable por estado y facultad; cambio de estado requiere comentario obligatorio ≥ 20 caracteres; notificación al organizador con la decisión; cambio registrado en log de auditoría (RF-29); verificado con pytest .	STK-05 pytest
RF-25	Mostrar dashboards institucionales con métricas globales.	S	Con datos de al menos 5 eventos en distintas facultades: total de eventos por facultad, inscritos vs. confirmados, tasa de ocupación, distribución por modalidad; filtros coherentes entre sí; latencia ≤ 1 min respecto a la BD; verificado con pytest contra dataset de prueba.	STK-05, STK-06 pytest
RF-26	Gestionar usuarios, asignar roles y configurar permisos (RBAC).	M	Roles disponibles: organizador, asistente, ponente, revisor, administrador; verificación del rol en servidor para cada operación sensible; intento con rol insuficiente retorna 403; cambio de rol registrado en log de auditoría; verificado con pytest para escalada de privilegios.	STK-05, STK-06 pytest
RF-27	Configurar categorías de eventos y plantillas reutilizables.	C	Catálogo de categorías gestionable por el administrador sin modificar código; plantillas disponibles para los organizadores al crear un evento; al eliminar una categoría en uso, el sistema advierte y requiere reasignación.	STK-05 pytest

ID	Descripción	M	Criterio de aceptación	STK / Verif.
RF-28	Integrar autenticación con SSO institucional mediante OAuth 2.0 / OpenID Connect (mock en POC).	M	Inicio de sesión delegado sin contraseña local; token validado en cada petición autenticada; token inválido o expirado retorna 401; el adaptador mock es reemplazable por la integración real sin modificar la lógica de negocio; verificado con pytest (token válido, inválido, expirado).	STK-01-06 pytest
RF-29	Registrar auditoría inmutable de acciones críticas.	M	Para cada acción listada (creación/cancelación de eventos, confirmación de pago, envíos masivos, cambio de rol, acceso a PII): entrada con usuario, timestamp UTC, tipo de acción, entidad afectada, resultado y trace_id ; entradas no modificables ni eliminables (intento de DELETE retorna 405); consultables por el administrador con filtros; verificado con pytest (escritura y prueba negativa de inmutabilidad).	STK-05, STK-06 pytest
RF-30	Búsqueda de eventos: autocompletado textual y búsqueda semántica con embeddings vectoriales (pgvector).	M	<i>Búsqueda textual</i> : sugerencias presentadas tras escribir 2 o más caracteres por coincidencia sobre título y descripción; sin resultados, se muestra mensaje descriptivo. <i>Búsqueda semántica</i> : cada evento publicado genera un embedding asociado a su título y descripción, almacenado en columna vector de PostgreSQL (pgvector); la consulta semántica retorna los k eventos más cercanos por similitud coseno, habilitando búsqueda contextual y recomendaciones; se demuestra el flujo completo (generación de embedding, almacenamiento, consulta <->) con el dataset de prueba. Todo verificado con pytest .	STK-02 pytest

Tabla 6: Catálogo de requisitos funcionales.

5. Requisitos no funcionales (RNF)

Definimos 18 RNF medibles y verificables, distribuidos por atributo de calidad. La sección comienza con **observabilidad** porque es el eje arquitectónico del proyecto: la regla de negocio RN-10 establece que ningún módulo *Must have* se considera implementado si no emite las señales de telemetría correspondientes.

Cuando un criterio menciona **pytest**, la verificación se realiza con la suite de pruebas integrada al pipeline CI en GitHub Actions. Las pruebas de carga se realizan con **Locust** ejecutado localmente o en un runner dedicado.

5.1. Observabilidad

Adoptamos el modelo de los **tres pilares** (logs estructurados, métricas y trazas) sobre OpenTelemetry como capa de instrumentación neutral. Esto desacopla la instrumentación del backend de observabilidad y nos permite evolucionar el stack (Prometheus, Grafana, Loki, Jaeger/Tempo) sin tocar el código.

ID	Descripción	Métrica / objetivo	Criterio de aceptación
RNF-01	Emisión de logs estructurados de los eventos operacionales y de seguridad críticos, instrumentados con OTel.	Cobertura y estructura de logging	(a) Logs en JSON con campos mínimos: <code>timestamp</code> (ISO 8601 UTC), <code>level</code> , <code>service</code> , <code>trace_id</code> , <code>span_id</code> , <code>user_id</code> (cuando aplique) y <code>message</code> . (b) Eventos obligatorios: inscripción, pago, generación de certificado, envío masivo, autenticación, cambio de rol y acceso a PII. (c) Logs consumibles por Loki (validable levantando el stack local en <code>docker-compose</code>). (d) Verificación mediante <code>pytest</code> que recoge los logs y valida el formato.
RNF-02	Exposición de métricas en tiempo real siguiendo el método RED.	Compleitud de señales RED por servicio	(a) Endpoint <code>/metrics</code> compatible con scraping de Prometheus. (b) Por endpoint instrumentado: <code>Rate</code> , <code>Errors</code> y <code>Duration</code> (histograma p50/p95/p99). (c) Métricas adicionales: tamaño de cola de correo, cupos disponibles por evento (gauge), duración de generación de certificados. (d) Dashboard de Grafana con señales RED de inscripciones y pagos incluido en el repositorio como artefacto. (e) Verificación con <code>pytest</code> que consulta <code>/metrics</code> y valida los nombres de métricas.
RNF-03	Trazabilidad distribuida de extremo a extremo mediante identificadores propagados entre componentes.	Compleitud de la cadena de trazas	(a) Instrumentación OTel en servicio web, procesador de cola y módulo de pagos. (b) Propagación conforme a W3C Trace Context (<code>traceparent</code> / <code>tracestate</code>). (c) <code>trace_id</code> presente en la respuesta HTTP (<code>X-Trace-Id</code>), en los logs y en el backend de trazas (Jaeger o Tempo). (d) Dado un <code>trace_id</code> , un operador puede reconstruir la cadena de spans en ≤ 5 min. (e) Verificado con prueba E2E: <code>pytest</code> inyecta un error deliberado, captura el <code>trace_id</code> y valida que aparece en el log y en el backend de trazas.
RNF-04	Endpoints de salud que reflejan la disponibilidad de las dependencias.	Disponibilidad y cobertura de health checks	(a) <code>/health/live</code> retorna 200 si el proceso está activo, 503 si no. (b) <code>/health/ready</code> verifica BD, cola y proveedor de correo; retorna estado por dependencia. (c) Tiempo de respuesta ≤ 500 ms en condiciones normales. (d) Verificado con <code>pytest</code> que detiene una dependencia y comprueba que <code>/health/ready</code> refleja el fallo en ≤ 10 s.
RNF-05	SLI/SLO documentados para los servicios críticos.	Existencia y coherencia de SLI/SLO	SLIs y SLOs documentados en el SAD para inscripciones, webhooks de pago y notificaciones; cada SLO incluye ventana de medición, umbral objetivo y herramienta de monitoreo; los SLOs son coherentes con las métricas obtenidas en las pruebas de Locust ejecutadas en el repositorio.

5.2. Rendimiento

Distinguimos dos escenarios de carga relevantes para el POC: **(a)** navegación del catálogo (lecturas sin escritura de cupos) y **(b)** pico de inscripción (lecturas más escrituras atómicas de cupos). Los criterios aplican exclusivamente a la lógica interna de la PGEA; la latencia de servicios externos queda excluida.

ID	Descripción	Métrica	Criterio de aceptación
RNF-06	Tiempo de respuesta del catálogo bajo carga típica.	p95 de latencia	$p95 \leq 500$ ms para listado con 100 eventos activos y 50 usuarios concurrentes en escenario (a). Validado con script <code>Locust</code> incluido en el repositorio; resultados exportados como artefacto del pipeline CI.
RNF-07	Tiempo de respuesta de inscripción gratuita bajo carga.	p95 de latencia	$p95 \leq 2$ s desde el envío del formulario hasta la pantalla de confirmación, con 20 usuarios concurrentes en escenario (b). El tiempo del correo asíncrono no se incluye (es trazable vía log, RNF-01). Validado con <code>Locust</code> .

Tabla 8: RNF — Rendimiento.

5.3. Escalabilidad

ID	Descripción	Métrica	Criterio de aceptación
RNF-08	Concurrencia en control de cupos sin sobreventa.	Atomicidad bajo concurrencia	50 intentos de inscripción concurrentes al último cupo disponible: solo uno se confirma; los demás reciben 409 con <code>no_capacity</code> ; sin pérdida de datos ni cupos negativos. Validado con prueba específica en <code>Locust</code> más <code>pytest</code> de verificación de consistencia.

Tabla 9: RNF — Escalabilidad.

5.4. Disponibilidad

ID	Descripción	Métrica	Criterio de aceptación
RNF-09	Recuperación ante fallo de dependencias y disponibilidad del POC.	Detección y recuperación	El sistema detecta fallos de dependencias en ≤ 10 s vía <code>/health/ready</code> ; ante reinicio de contenedor, el servicio vuelve a estado <code>ready</code> en ≤ 30 s; verificado con <code>pytest</code> que detiene una dependencia y la reinicia, midiendo los tiempos. En la demo, el operador puede provocar un fallo y verificar el comportamiento en el dashboard.

Tabla 10: RNF — Disponibilidad.

5.5. Seguridad y privacidad

ID	Descripción	Métrica	Criterio de aceptación
RNF-10	Cumplimiento operacional de la Ley 1581 mediante mecanismos verificables.	Conformidad legal + funcionalidades	(a) Política de privacidad accesible desde el flujo de registro. (b) Consentimiento informado registrado con timestamp por usuario. (c) Endpoint que permite al usuario consultar sus datos personales almacenados. (d) Endpoint que permite solicitar la supresión de datos; en el POC, la supresión anonimiza los campos PII manteniendo trazabilidad de auditoría. (e) Log de acceso a PII consultable por el administrador con usuario, timestamp y operación. Verificado con pytest (consulta, supresión y log de acceso).
RNF-11	Cifrado en tránsito y autenticación delegada.	Cifrado y manejo de credenciales	TLS 1.2+ en todas las comunicaciones usuario-sistema (terminado en Nginx); autenticación delegada a SSO (sin contraseñas locales); tokens de sesión con expiración configurable; verificado con inspección de configuración Nginx + pytest de expiración de token.
RNF-12	Control de acceso por roles (RBAC) en operaciones sensibles.	Efectividad de la matriz de permisos	Matriz RBAC documentada; validación de rol en servidor para cada operación sensible; sin posibilidad de escalada desde el cliente; verificado con pytest con pruebas negativas que intentan operaciones con rol insuficiente y validan respuesta 403.

Tabla 11: RNF — Seguridad y privacidad.

5.6. Usabilidad

ID	Descripción	Métrica	Criterio de aceptación
RNF-13	Interfaz funcional y responsive en desktop y móvil.	Compatibilidad de viewport	La interfaz se usa correctamente en viewports ≥ 360 px de ancho (desktop y móvil estándar); los flujos principales (descubrir evento, inscribirse, ver agenda) se completan en ambos viewports; verificado con prueba manual en el navegador con emulador de dispositivo.

Tabla 12: RNF — Usabilidad.

5.7. Interoperabilidad

ID	Descripción	Métrica	Criterio de aceptación
RNF-14	SSO delegado mediante adaptador desacoplado del proveedor.	Estandarización y desacople	Inicio de sesión con OAuth 2.0 / OpenID Connect; el adaptador mock del POC implementa la misma interfaz que el adaptador real previsto; sustituir un adaptador por otro no requiere cambios en la lógica de negocio; verificado con <code>pytest</code> usando ambos adaptadores sobre los mismos casos de prueba.
RNF-15	Adaptadores intercambiables para servicios externos (pasarela, correo).	Desacople de integraciones	Cada integración externa expone una interfaz Python única; cambiar el proveedor (mock → real) se realiza únicamente vía configuración por variables de entorno sin modificar la lógica de negocio; verificado con <code>pytest</code> usando implementaciones mock e implementaciones de prueba que validan la interfaz.

Tabla 13: RNF — Interoperabilidad.

5.8. Modificabilidad

ID	Descripción	Métrica	Criterio de aceptación
RNF-16	Extensibilidad por configuración y separación entre dominio e infraestructura.	Cambios localizados	Agregar una categoría de evento se realiza sin modificar código (vía datos); cambiar el proveedor de correo se hace solo en la capa de adaptadores; verificado documentalmente mediante el árbol de paquetes y por un cambio demostrable de adaptador (<code>pytest</code> con dos implementaciones de la misma interfaz).

Tabla 14: RNF — Modificabilidad.

5.9. Testeabilidad

ID	Descripción	Métrica	Criterio de aceptación
RNF-17	Cobertura de pruebas automatizadas en los módulos críticos.	Cobertura de código (pytest)	Cobertura ≥ 70 % en los módulos de inscripciones, pagos y autenticación, medida con <code>coverage.py</code> ; reporte generado en cada ejecución del pipeline CI en GitHub Actions; el pipeline falla si la cobertura cae por debajo del umbral.
RNF-18	Pruebas de carga reproducibles para validar RNF de rendimiento y escalabilidad.	Ejecutabilidad de scripts Locust	Scripts Locust para los escenarios de RNF-06, RNF-07 y RNF-08 incluidos en el repositorio; ejecutables localmente con docker-compose contra el POC; resultados (CSV con percentiles) exportables como artefacto del pipeline.

Tabla 15: RNF — Testeabilidad.

6. Reglas de negocio

Cada regla es una política invariable del dominio que el sistema debe respetar. Se vinculan a los requisitos funcionales y no funcionales impactados.

ID	Regla	Justificación	Impacto
RN-01	Una inscripción es válida solo si el evento tiene cupo disponible en el momento de la confirmación. La verificación y el decremento del cupo se ejecutan como operación atómica respecto a cualquier otra inscripción concurrente.	Evita sobreventa; garantiza consistencia ante concurrencia.	RF-03, RF-04; RNF-08
RN-02	La cancelación de inscripción y la liberación del cupo siguen la política de cancelación del evento. Los reembolsos en eventos de pago dependen del estado reportado por el adaptador de pasarela.	Flexibilidad al organizador y consistencia con el flujo de pagos.	RF-17; RNF-15
RN-03	Un evento es visible en el catálogo solo cuando su estado es publicado . Los borradores, pendientes de aprobación o cancelados no son visibles a los asistentes.	Evita exponer borradores al público.	RF-01, RF-06
RN-04	Solo el organizador dueño del evento (o un administrador) puede editarlo, enviar comunicaciones masivas e iniciar la generación de certificados. Solo usuarios con inscripción confirmada acceden al material del evento.	Seguridad y responsabilidad por rol.	RF-06, RF-12, RF-13, RF-18; RNF-12

ID	Regla	Justificación	Impacto
RN-05	Un certificado de asistencia se emite solo si existe registro de asistencia confirmado para ese usuario y evento. Los certificados incluyen código único verificable.	Integridad de la acreditación; prevención de fraude.	RF-13, RF-19, RF-21
RN-06	Una inscripción a un evento de pago queda confirmada solo cuando el sistema recibe confirmación explícita del adaptador de pasarela. Hasta entonces el estado es pendiente_pago ; el cupo se reserva temporalmente con timeout configurable (15 min por defecto). Al expirar sin confirmación, el cupo se libera.	Evita confirmar inscripciones sin pago efectivo.	RF-04
RN-07	El tratamiento de datos personales se limita al propósito declarado, conforme a la Ley 1581 de 2012: (a) consentimiento informado registrado con timestamp; (b) el usuario puede consultar sus datos personales en cualquier momento; (c) el usuario puede solicitar la supresión de sus datos (los datos no vinculados a obligaciones legales son anonimizados en ≤ 30 días desde la aprobación); (d) el acceso a PII queda registrado en el log de auditoría con identificación del actor y trace_id ; (e) no se transfieren datos a terceros distintos de los proveedores declarados sin consentimiento adicional.	Cumplimiento normativo y operacionalización verificable de la Ley 1581.	RF-26, RF-29; RNF-10
RN-08	Los envíos masivos de correo se realizan de forma asíncrona mediante una cola, respetando el límite de tasa del proveedor configurado. El hilo principal del servidor no debe bloquearse durante el envío.	Evita saturar el proveedor y bloquear la interfaz.	RF-12, RF-15
RN-09	El enlace de acceso a un evento virtual es visible exclusivamente para usuarios con inscripción confirmada . No se incluye en comunicaciones masivas no autenticadas.	Control de acceso a contenido exclusivo.	RF-18; RNF-12

RN-10	La instrumentación de observabilidad (logs, métricas, trazas) es una responsabilidad transversal no opcional. Ningún módulo <i>Must have</i> se considera implementado si no emite las señales de log (RNF-01) y el <code>trace_id</code> de correlación (RNF-03) asociados a sus operaciones críticas. Aplica a los módulos de inscripción, webhooks de pago, generación de certificados, envío de correo y autenticación.	La observabilidad es el eje arquitectónico del proyecto y la base de la verificabilidad operacional.	RNF-01, RNF-02, RNF-03; todos los RF Must
-------	---	--	---

Tabla 16: Reglas de negocio del sistema PGEA.

7. Casos de uso

7.1. CU-01: Inscribirse a un evento

Actores: Asistente (STK-02).

Precondición: Usuario autenticado; evento publicado con cupo > 0 .

Flujo principal:

1. El asistente accede al catálogo y selecciona un evento (RF-01, RF-02).
2. Revisa la información del evento.
3. Selecciona “Inscribirse”. Si el evento tiene formulario personalizado (RF-08), lo completa.
4. Si es gratuito: el sistema registra la inscripción en **confirmada**, decrementa el cupo atómicamente (RN-01) y encola el correo de confirmación (RF-15). El sistema emite log con `trace_id` (RN-10).
5. Si es de pago: el sistema reserva el cupo en **pendiente_pago** y redirige al adaptador de pasarela (RF-04, RN-06).
6. Al confirmar el pago, el adaptador notifica vía webhook; el sistema transiciona a **confirmada** y envía correo (RF-15).

Flujos alternativos:

- **3a. Sin cupo:** 409 con `no_capacity`.
- **3b. Duplicado:** 409 con `duplicate_registration`.
- **5a. Timeout de pago:** el cupo reservado se libera (RN-06) y el asistente recibe notificación.
- **5b. Pago rechazado:** el cupo se libera y se notifica al asistente con instrucciones para reintentar.

Postcondición: Inscripción **confirmada**; correo encolado; cupo actualizado; log de auditoría registrado.

7.2. CU-02: Crear y publicar un evento

Actores: Organizador (STK-01).

Precondición: Usuario con rol organizador autenticado.

Flujo principal:

1. Accede al panel y selecciona “Crear nuevo evento”.
2. Completa: título, descripción, fecha/hora inicio y fin, modalidad, ubicación o enlace, cupo máximo y categoría (RF-05).

3. Configura el tipo de inscripción (RF-07) y, opcionalmente, la agenda (RF-09).
4. El sistema guarda el evento en **borrador** con el **id** asignado.
5. El organizador revisa la previsualización y selecciona “Publicar”.
6. Si requiere aprobación (RF-24): el evento pasa a **pendiente_aprobación** y se notifica al administrador.
7. Una vez aprobado (o sin requerir aprobación): el evento pasa a **publicado**.

Flujos alternativos:

- **2a. Campo obligatorio vacío o fechas incoherentes:** 400 con mensaje específico; el formulario conserva los datos ya completados.
- **7a. Aprobación rechazada:** el administrador proporciona un comentario obligatorio; el evento permanece en **borrador**.

Postcondición: Evento publicado visible en el catálogo; acción registrada en log de auditoría (RF-29).

7.3. CU-03: Enviar comunicación masiva a inscritos

Actores: Organizador (STK-01).

Precondición: Evento con al menos un inscrito confirmado; organizador dueño.

Flujo principal:

1. Accede al panel del evento y selecciona “Comunicaciones”.
2. Redacta el mensaje y define la segmentación (RF-12).
3. Confirma el envío. El sistema encola el mensaje de forma asíncrona (RN-08).
4. Muestra el progreso: enviados, fallidos y pendientes.
5. Al completarse, el organizador recibe notificación con el resumen.

Flujos alternativos:

- **3a. Fallo del proveedor:** los mensajes fallidos se reencolan para reintento; el operador consulta la salud de la cola vía RNF-02.
- **2a. Límite de tasa:** la cola espacia los envíos para respetar el límite.

Postcondición: Correos entregados a la mayor parte posible de destinatarios; estado por destinatario registrado; log de la operación con **trace_id** disponible.

7.4. CU-04: Descargar certificado digital

Actores: Asistente (STK-02), Ponente (STK-03).

Precondición: Evento finalizado; usuario con asistencia registrada (RN-05).

Flujo principal:

1. Accede a “Mis certificados” o al panel del evento.
2. Selecciona el certificado.
3. El sistema verifica la asistencia registrada (RF-19) y el estado **confirmada** (RN-05).
4. El sistema retorna el PDF para descarga.
5. El usuario descarga el PDF con nombre, evento, fecha, tipo de certificado y código único verificable (RF-13).
6. El código puede validarse en la URL pública del sistema.

Flujos alternativos:

- **3a. Sin asistencia registrada:** 403 con mensaje que informa la condición faltante.

- **4a. Error de generación:** el sistema registra el fallo con `trace_id` (RN-10); el usuario recibe mensaje con la referencia para reportar el incidente.

Postcondición: PDF descargado; log de acceso y generación registrado (RF-29).

7.5. CU-05: Gestionar invitación a ponente

Actores: Organizador (STK-01), Ponente (STK-03).

Precondición: Evento creado; correo del ponente disponible.

Flujo principal:

1. El organizador agrega el correo del ponente en la sección “Ponentes” (RF-10).
2. El sistema genera un token de invitación de un solo uso y envía el correo (encolado asíncronamente).
3. El ponente accede al enlace y selecciona “Aceptar participación”.
4. Completa su perfil: nombre, fotografía y biografía (RF-22).
5. Sube su material de presentación (RF-22).
6. El sistema notifica al organizador.

Flujos alternativos:

- **3a. Ponente declina:** el organizador recibe notificación y reasigna la sesión.
- **5a. Formato no aceptado o tamaño excedido:** error con los formatos permitidos.
- **2a. Token expirado o ya utilizado:** 410 con indicación de contactar al organizador.

Postcondición: Participación confirmada o declinada registrada; perfil y material visibles en la ficha del evento si está confirmado; organizador notificado.

7.6. CU-06: Supervisión operacional y diagnóstico

Actores: Administrador (STK-05) / Operador de infraestructura.

Precondición: Sistema desplegado y stack de observabilidad activo.

Flujo principal:

1. El operador consulta el dashboard de Grafana con las señales RED de los servicios críticos (RNF-02).
2. Detecta una elevación en la tasa de errores del servicio de inscripciones.
3. Consulta los logs en Loki filtrando por servicio y nivel `ERROR` en el rango afectado (RNF-01).
4. Identifica el `trace_id` de una solicitud fallida.
5. Usa ese `trace_id` en Jaeger/Tempo para reconstruir la cadena de spans y localizar el componente donde ocurrió el fallo (RNF-03).
6. Resuelve el problema y verifica que la tasa de errores en el dashboard vuelve al umbral normal.

Flujos alternativos:

- **1a. /health/ready retorna dependencia fallida:** el operador identifica el componente degradado directamente sin necesidad de revisar logs (RNF-04, RNF-09).

Postcondición: Incidente localizado y resuelto; causa raíz trazada en ≤ 5 min desde el `trace_id` reportado (RNF-03).

7.7. CU-07: Aprobación institucional de eventos

Actores: Administrador (STK-05).

Precondición: Al menos un evento en `pendiente_aprobación`.

Flujo principal:

1. El administrador accede al panel institucional con la lista global filtrable por estado y facultad (RF-24).
2. Selecciona un evento pendiente y revisa sus detalles.
3. Toma la decisión (aprobar o rechazar) e ingresa el comentario obligatorio.
4. El sistema actualiza el estado, notifica al organizador y registra la acción en el log de auditoría con `trace_id` (RF-29).
5. El administrador consulta el dashboard institucional (RF-25).

Flujos alternativos:

- **3a. Comentario insuficiente:** el sistema exige al menos 20 caracteres.

Postcondición: Estado del evento actualizado; organizador notificado; acción auditada; métricas institucionales actualizadas.

8. Riesgos de implementación

Identificamos cinco riesgos técnicos de mayor impacto para la entrega final. Su documentación nos sirve tanto para priorizar decisiones de diseño como para evidenciar qué parte del alcance es realmente comprometida y dónde aplicamos mitigaciones concretas.

ID	Riesgo	Nivel	Afecta	Estrategia de mitigación
RI-01	Concurrencia en control de cupos. Condiciones de carrera en la operación de verificación y decremento atómico bajo carga de inscripción simultánea.	A	RF-03, RF-04; RNF-08	Usamos transacciones con bloqueo pesimista en PostgreSQL desde el primer sprint de implementación; diseñamos una prueba específica en Locust con 50 escrituras concurrentes al último cupo y un <code>pytest</code> que verifica que no hay sobreventa.
RI-02	Integración asíncrona con pasarela de pagos. El ciclo del webhook (recepción, idempotencia, timeout, liberación de cupo) involucra múltiples estados y casos de error difíciles de probar sin un sandbox estable.	A	RF-04; RNF-15	En el POC implementamos un adaptador mock que simula los cuatro estados del webhook (confirmada, expirada, fallida y duplicada); el código del adaptador real es intercambiable por configuración. Probamos los cuatro estados con <code>pytest</code> .
RI-03	Saturación de la cola de correo. Sin diseño asíncrono adecuado, los envíos masivos pueden bloquear el servidor o perder mensajes sin trazabilidad.	M	RF-12, RF-15	La cola se implementa como componente desacoplado desde el primer sprint, instrumentada con métricas de tamaño y tasa de error (RNF-02). En el POC el adaptador mock registra cada envío en logs estructurados, lo que nos permite validar el flujo sin proveedor real.

ID	Riesgo	Nivel	Afecta	Estrategia de mitigación
RI-04	Instrumentación de observabilidad incompleta. Si la instrumentación con OTel no se introduce desde el inicio, queda como capa posterior generando brechas de cobertura.	M	RNF-01-05; RN-10	Configuramos el SDK de OTel y el pipeline de exportación (OTLP → Prometheus / Loki / Jaeger) en el primer sprint, antes de implementar la lógica de negocio. La regla RN-10 hace explícito que ningún módulo Must se cierra sin emitir las señales requeridas.
RI-05	Cumplimiento operacional de la Ley 1581. Los mecanismos de consulta, supresión y auditoría son fáciles de diferir y convertir en deuda técnica.	M	RNF-10; RN-07	Diseñamos los endpoints de consulta y supresión desde el modelo de datos; incluimos un caso de prueba en <code>pytest</code> que verifica que tras una supresión, los campos de PII de los registros del usuario están anonimizados pero el log de auditoría persiste correctamente.

Tabla 17: Riesgos de implementación. Nivel: A = Alto, M = Medio.

9. Modelo de datos conceptual

9.1. Diagrama Entidad-Relación

El siguiente diagrama representa las entidades del dominio y sus relaciones a nivel conceptual. No es el modelo físico; los tipos específicos, índices (incluyendo el índice de `pgvector` sobre embeddings de eventos) y claves foráneas se especifican en el SAD.

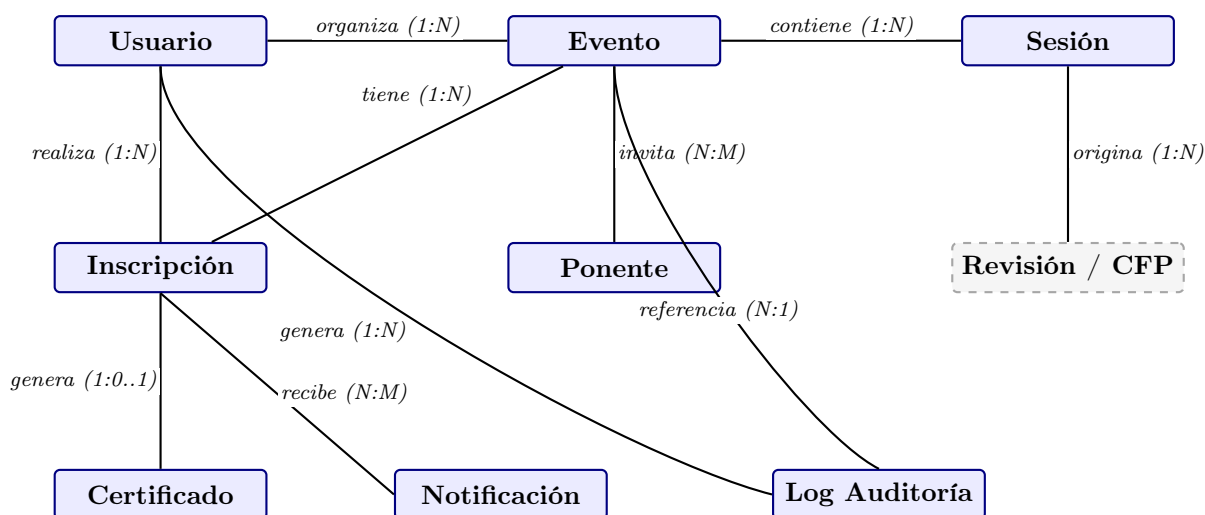


Figura 1: Modelo de datos conceptual de la PGEA. Borde discontinuo indica funcionalidad *Won't have* en el POC.

9.2. Descripción de entidades principales

- **Usuario:** actor del sistema (organizador, asistente, ponente, revisor o administrador). Identificado por correo institucional o token de invitación externo.

- **Evento:** unidad central; tiene ciclo de vida (borrador → pendiente → publicado → cancelado). Almacena su embedding vectorial (**pgvector**) para búsqueda semántica (RF-30).
- **Sesión:** unidad de agenda dentro de un evento; puede tener ponente asignado y track asociado.
- **Inscripción:** vincula Usuario y Evento; ciclo de estados (pendiente_pago → confirmada → cancelada → expirada).
- **Ponente:** relación N:M entre Usuario y Evento con estado (invitado, confirmado, declinado), perfil y material de presentación.
- **Certificado:** generado por el sistema cuando la inscripción tiene asistencia confirmada; incluye código único verificable.
- **Notificación:** registro de cada comunicación enviada o intentada, con estado por destinatario y **trace_id** de la operación de origen.
- **Log Auditoría:** tabla append-only de acciones críticas; no modificable por ningún rol de usuario; incluye **trace_id** para correlación con observabilidad.
- **Revisión / CFP** (entidad débil): propuesta enviada a revisión por pares; funcionalidad *Won't have* en el POC; se documenta el modelo conceptual para su evolución futura.

10. Matriz de trazabilidad

10.1. Trazabilidad primaria: Stakeholder → Requisito

La tabla siguiente vincula cada RF y bloque de RNF con los stakeholders que lo originan o se benefician directamente. ✓ indica relación directa.

ID	Requisito	Org.	Asis.	Pon.	Rev.	Adm.	Univ.
RF-01	Catálogo con filtros		✓				
RF-02	Ordenar eventos		✓				
RF-03	Inscripción gratuita		✓				
RF-04	Inscripción paga (mock)		✓			✓	✓
RF-05	Crear evento	✓					
RF-06	Editar/publicar/cancelar	✓				✓	
RF-07	Tipos de inscripción	✓	✓				
RF-08	Formulario personalizado	✓	✓				
RF-09	Agenda y sesiones	✓	✓	✓			
RF-10	Invitar ponentes	✓		✓			
RF-11	Lista de inscritos + CSV	✓					
RF-12	Comunicaciones masivas (mock)	✓					
RF-13	Generar certificados	✓	✓	✓			✓
RF-14	Reportes del evento	✓					
RF-15	Confirmaciones automáticas		✓				
RF-16	Exportar a calendario		✓				
RF-17	Cancelar inscripción		✓				
RF-18	Acceder al material		✓	✓			
RF-19	Registrar asistencia	✓	✓				
RF-20	Evaluación post-evento		✓			✓	✓
RF-21	Descarga de certificado		✓	✓			
RF-22	Perfil y material del ponente	✓		✓			
RF-23	CFP / Revisión por pares (W)				✓		
RF-24	Aprobación admin					✓	✓

ID	Requisito	Org.	Asis.	Pon.	Rev.	Adm.	Univ.
RF-25	Dashboard institucional					✓	✓
RF-26	Gestión de usuarios y RBAC					✓	✓
RF-27	Categorías y plantillas	✓				✓	✓
RF-28	Auth SSO (mock)	✓	✓	✓	✓	✓	✓
RF-29	Auditoría inmutable					✓	✓
RF-30	Búsqueda textual + semántica pgvector (M)		✓				
RNF-01-05	Observabilidad	✓	✓			✓	✓
RNF-06-07	Rendimiento	✓	✓			✓	✓
RNF-08	Escalabilidad	✓	✓			✓	✓
RNF-09	Disponibilidad	✓	✓			✓	✓
RNF-10-12	Seguridad y privacidad	✓	✓	✓	✓	✓	✓
RNF-13	Usabilidad	✓	✓	✓	✓		
RNF-14-15	Interoperabilidad	✓	✓			✓	✓
RNF-16	Modificabilidad					✓	✓
RNF-17-18	Testeabilidad					✓	✓

Tabla 18: Trazabilidad Stakeholder → Requisito.

10.2. Trazabilidad extendida de requisitos críticos

La tabla siguiente encadena los requisitos de mayor impacto arquitectónico: Actor → Necesidad (STK) → RF → RN → CU → Criterio de aceptación clave → Componente arquitectónico.

Actor	Necesidad	RF	RN	CU	Criterio clave	Componente
Asistente	Control de cupos (STK-02)	RF-03, RF-04	RN-01, RN-06	CU-01	50 inscripciones concurrentes sin sobreventa; mock de pasarela cubre los cuatro estados.	Módulo de Inscripciones + Adaptador de Pasarela + BD con bloqueo pesimista
Organizador	Comunicaciones masivas (STK-01)	RF-12, RF-15	RN-08	CU-03	Envío encolado; hilo principal no bloqueado; métricas de cola en dashboard.	Servicio de Correo (adaptador) + Cola + Métricas OTel
Admin.	Supervisión operacional (STK-05)	RF-24, RF-25, RF-29	RN-04, RN-10	CU-07, CU-06	Dashboard con señales RED; auditoría con <code>trace_id</code> ; incidente localizable en ≤ 5 min.	Dashboard + Pipeline OTel + Log de auditoría (append-only)
Asistente	Certificados digitales (STK-02)	RF-13, RF-19, RF-21	RN-05	CU-04	Generación con código verificable único; log de generación trazable por <code>trace_id</code> .	Generador PDF + Almacenamiento + Instrumentación OTel

Actor	Necesidad	RF	RN	CU	Criterio clave	Componente
Usuario	Protección de datos (STK-06)	RF-26, RF-29	RN-07	—	Supresión procesada en ≤ 30 días; log de acceso a PII; consentimiento con timestamp.	Gestión de Usuarios + Log de acceso a PII + Flujo de anonimización
Sistema	Observabilidad transversal (STK-05, STK-06)	Todos los Must	RN-10	CU-06	Stack OTel desde sprint 1; dashboards RED entregados; trazas retenidas ≥ 7 días.	OTel SDK + OTLP + Prometheus + Grafana/Loki + Jaeger/Tempo
Organizador	Gestión de eventos (STK-01)	RF-05, RF-06, RF-09	RN-03, RN-04	CU-02	Flujo crear $\rightarrow$ publicar verificado E2E; cambios reflejados en ≤ 10 s; 403 a terceros.	Módulo de Eventos (CRUD) + Motor RBAC + BD principal
Asistente	SSO institucional (STK-02)	RF-28	—	CU-01, CU-02	Mock OAuth 2.0 con misma interfaz que el adaptador real; sustitución sin cambios en lógica de negocio.	Servicio de Autenticación + Adaptador OAuth (mock $\rightarrow$ real)

Tabla 19: Trazabilidad extendida de requisitos críticos.

11. Criterios de aceptación del sistema

El sistema se considera aceptado cuando se cumplen todas las condiciones siguientes:

1. El 100 % de los RF clasificados como *Must have* están implementados y pasan las pruebas de aceptación de sus criterios individuales (verificadas con `pytest` en el pipeline CI).
2. Los RNF de rendimiento (RNF-06 y RNF-07) y escalabilidad (RNF-08) están validados mediante scripts de `Locust` incluidos en el repositorio: $p95 \leq 500$ ms en catálogo con 50 usuarios concurrentes; $p95 \leq 2$ s en inscripción con 20 usuarios concurrentes; 50 inscripciones concurrentes al último cupo sin sobreventa.
3. Los siete casos de uso documentados funcionan end-to-end en la demostración en vivo.
4. El stack de observabilidad (RNF-01–05) está operacional en el ambiente de la demo: logs estructurados JSON con `trace_id` visibles en Loki; métricas RED consultables vía Prometheus; al menos un dashboard de Grafana de los servicios de inscripción incluido como artefacto; endpoints `/health/live` y `/health/ready` responden correctamente; trazas visibles en Jaeger o Tempo.
5. El cumplimiento de Ley 1581 (RNF-10, RN-07) es demostrable: política de privacidad accesible; endpoints de consulta y supresión funcionales; log de acceso a PII consultable por administrador.
6. Las reglas RN-01 (atomicidad de cupos), RN-05 (emisión de certificados) y RN-06 (confirmación de pago) están cubiertas por pruebas de integración automatizadas en el pipeline.
7. El código está en el repositorio Git con historial coherente; el sistema es ejecutable siguiendo el README; el pipeline CI/CD en GitHub Actions ejecuta `pytest` en cada push a `main` y `stage`; el despliegue en AWS Lightsail (Docker + Nginx) está documentado; la cobertura de `pytest` ≥ 70 % en los módulos de inscripciones, pagos y autenticación es medida con `coverage.py` y reportada en el pipeline; los SLI/SLO del SAD (RNF-05) son coherentes con las métricas obtenidas en las pruebas de `Locust`.